

ENGINEERING MEMO

ExamGate: Deadline-Night Submission Platform

Design choices, complexity summary, and measured scaling

To	ExamGate Team Lead
From	Mehmet Arin Atasayar (210444088)
Course	CENG 201 - Data Structures and Algorithms
Subject	Why I chose the data structures used in ExamGate
GitHub	https://github.com/mehmetarinatasayar/CENG201_TermProject

1. Purpose and result

I built ExamGate as a console program that simulates the final hours before an assignment deadline. The main problem is not the screen; it is keeping uploads ordered, finding the active file for each student, supporting rollback, and marking late submissions exactly. I implemented the core structures with arrays and linked nodes instead of Java collection classes.

The completed program connects the generator, intake queue, priority dispatcher, registry, rollback service, AVL timeline, and report service. With seed **20260725**, the WP7 run generated 2,500 upload events for 800 students. The full-buffer rule accepted 2,400 and rejected 100. It counted 1,600 accepted re-uploads, 36 rollbacks, and 400 late uploads. Repeating the same run produced the same functional output.

My main design goal was to prevent work from becoming linear when the deadline traffic increases. The measurements on page 5 show this clearly: for 100,000 records, the circular queue completed in 2.932 ms while the shifting queue took 3,731.709 ms. The heap dispatcher completed in 14.686 ms while the sorted-array dispatcher took 15,742.819 ms.

Implemented pipeline

Seeded generator -> circular intake queue -> max-heap dispatcher -> hash registry -> AVL timeline -> sorting and search reports

Evidence	Verified result
Deadline rule	timestamp > 23:59:00.000; exactly 23:59 is on time
WP5 balance	10,000 increasing inserts: plain BST height 10,000, AVL height 14
WP6 report	Merge and insertion results matched; binary search found the first late record
Repeatability	Same seed and same WP7 checkpoint values

2. Design-decision table

Operation	Chosen structure	Alternatives	Time complexity	Why I chose it
studentId lookup	Hash table with separate chaining; resize at 0.75	Open addressing; sorted array	Expected $O(1)$; worst $O(n)$	Re-uploads need fast lookup. Chaining stays understandable and deletion is not needed. At 0.75 the chains stay short under normal hashes.
Full intake buffer	Circular queue; reject-new policy	Drop oldest upload	$O(1)$ enqueue and dequeue	Dropping the oldest would silently remove a file that was already accepted. Reject-new gives the latest student a visible failure and a chance to retry.
Grading priority	Max-heap key: accommodation first, then earlier time, then studentId	FIFO only; sorted array	$O(\log n)$ submit/poll; $O(n)$ build	The key makes the promised priority rule explicit. studentId gives deterministic order if two other fields tie.
Midnight burst	Bottom-up build-heap	Repeated heap submit	$O(n)$ build instead of $O(n \log n)$	Most nodes start close to the leaves and need little or no movement. This is a better fit for a whole burst.
Time index	AVL tree keyed by timestamp	Plain BST; sorted array	$O(\log n)$ insert; $O(\log n + m)$ range	Uploads arrive mostly in time order. A plain BST became height 10,000, but the AVL height was 14.
Time-sheet sort	Merge sort	Quick sort; insertion sort	$O(n \log n)$ worst case	Merge sort has a predictable worst case. I kept insertion sort only as a correctness cross-check and baseline.
Top-K files	Size-K min-heap	Sort every file by size	$O(N \log K)$	Only K results are needed. With $K=10$, keeping ten candidates avoids a full $N \log N$ sort.

Trade-offs I accepted

- Separate chaining can still become $O(n)$ if many keys land in one bucket. Expected $O(1)$ assumes the string hash distributes student IDs reasonably evenly.
- Reject-new hurts the student whose request reaches a full buffer, but it does not erase an earlier accepted request. I considered that the smaller and more visible failure.
- Merge sort uses $O(n)$ extra array space. I accepted this because its $O(n \log n)$ worst case is safer for a report that may already be in timestamp order.
- Version stacks use memory proportional to saved versions. In a real service I would cap the history per student, for example at 20 versions, while keeping push and rollback $O(1)$.

The fairness detail in WP3 is important: the heap's internal array is not FIFO, but *next()* is fair because every comparison uses the complete priority key. Therefore the root is always the best remaining submission according to the stated rule.

3. Big-O summary - data model and WP1-WP3

Here, n is the number of records in a structure, c is queue capacity, and k is the requested top-K size. Array allocation is shown as $O(c)$ where it creates a capacity-sized array.

Class / public operation	Expected / normal	Worst case	Short reason
Submission constructor, getters, isLate	$O(1)$	$O(1)$	A fixed number of fields
Submission.replaceFile / restoreFile	$O(1)$	$O(1)$	Update fixed fields
Submission.clock / toString	$O(1)^*$	$O(1)^*$	Fixed-format project fields
SubmissionRegistry constructor	$O(1)$	$O(1)$	Initial table size is constant
SubmissionRegistry.put	$O(1)$ amortized	$O(n)$	Short chain normally; resize sometimes
SubmissionRegistry.lookup	$O(1)$	$O(n)$	One bucket normally; one long chain worst
SubmissionRegistry.updateVersion	$O(1)$	$O(n)$	Lookup plus constant update
SubmissionRegistry.size	$O(1)$	$O(1)$	Stored counter
NaiveUploadQueue constructor	$O(c)$	$O(c)$	Allocate capacity array
NaiveUploadQueue.enqueue / size	$O(1)$	$O(1)$	Append or read counter
NaiveUploadQueue.dequeue	$O(n)$	$O(n)$	Shift remaining entries left
NaiveUploadQueue.printState	$O(n)$	$O(n)$	Print every waiting ID
CircularUploadQueue constructor	$O(c)$	$O(c)$	Allocate capacity array
CircularUploadQueue.enqueue / dequeue / size	$O(1)$	$O(1)$	Move wrapped indexes
CircularUploadQueue.printState	$O(n)$	$O(n)$	Print every waiting ID
NaiveDispatcher constructor / size	$O(1)$	$O(1)$	Constant initial array / counter
NaiveDispatcher.submit	$O(n)$	$O(n)$	Find place and shift entries
NaiveDispatcher.next	$O(1)$	$O(1)$	Remove the prepared last entry
HeapDispatcher constructor / size	$O(1)$	$O(1)$	Constant initial array / counter
HeapDispatcher.submit	$O(\log n)$	$O(\log n)$	Sift up tree height
HeapDispatcher.next	$O(\log n)$	$O(\log n)$	Sift down tree height
HeapDispatcher.loadBurst	$O(n)$	$O(n)$	Bottom-up build-heap

* The strings used by this assignment have bounded practical length. If arbitrary string length L is counted, formatting and hashing include an $O(L)$ term.

Why build-heap is linear

Although a single sift can be $O(\log n)$, only a few nodes are near the root and able to move that far. About half the nodes are leaves and do no work. Adding the work by node height gives $O(n)$, not $O(n \log n)$.

4. Big-O summary - WP4-WP7

Class / public operation	Normal	Worst case	Short reason
VersionRecord constructor / getters / toString	$O(1)$	$O(1)$	Fixed saved fields
VersionStack constructor / isEmpty / size	$O(1)$	$O(1)$	Head pointer and counter
VersionStack.push / pop	$O(1)$	$O(1)$	Add/remove linked-list head
VersionStack.printState	$O(v)$	$O(v)$	Print v saved versions
RollbackService constructor	$O(1)$	$O(1)$	Store registry reference
RollbackService.reupload	$O(1)$ expected	$O(n)$	Registry access plus stack push
RollbackService.rollback	$O(1)$ expected	$O(n)$	Registry access plus stack pop
RollbackService.printHistory	$O(v)$	$O(v+n)$	Registry lookup and history print
PlainSubmissionBST constructor	$O(1)$	$O(1)$	Empty root
PlainSubmissionBST.insert	$O(\log n)$	$O(n)$	Unbalanced height can become n
PlainSubmissionBST.height	$O(n)$	$O(n)$	Visits every node
SubmissionTimeline.insert	$O(\log n)$	$O(\log n)$	AVL height stays logarithmic
SubmissionTimeline.height / rotation counters	$O(1)$	$O(1)$	Stored root height / counters
SubmissionTimeline.submittedBetween	$O(\log n + m)$	$O(n)$	Prunes outside range; returns m matches
ReportService.topKLargest	$O(N \log k)$	$O(N \log k)$	Size-k min-heap sweep
ReportService.sortByTimeFast	$O(N \log N)$	$O(N \log N)$	Merge sort
ReportService.sortByTimeInsertion	$O(N^2)$	$O(N^2)$	Nested shifts; best case $O(N)$
ReportService.findFirstAfter	$O(\log N)$	$O(\log N)$	Binary search
ReportService.printSheet	$O(N)$	$O(N)$	Print all rows
ScenarioGenerator constructor	$O(S)$	$O(S)$	Create S-student roster flags
ScenarioGenerator.studentId / flag / nextUpload	$O(1)$	$O(1)$	Indexed or fixed work
ScenarioGenerator.generateUploadEvents	$O(E)$	$O(E)$	Generate E deterministic events
UploadEvent constructor / getters	$O(1)$	$O(1)$	Fixed fields
ExamGateEngine.run	$O(E \log E)$ typical	$O(E \log E + E^2)^*$	Pipeline plus reporting

* The integrated engine uses the designed queue, heap, hash table, AVL tree, and merge sort. The E^2 term is only a conservative bound if hash chains or other inputs become pathological; with the generated IDs, the measured behavior follows the designed costs.

Range-query argument

The recursive AVL query skips a left subtree when its keys are too small and skips a right subtree when its keys are too large. It follows search paths of logarithmic height and must also visit/output the m matches, so the intended cost is $O(\log n + m)$.

5. Scaling study and conclusion

I measured only the data-structure work with **System.nanoTime()**. Record generation happened before each timed part. I also ran a 1,000-record warm-up and discarded it before collecting the table, as required. Times depend on the machine, so the growth pattern matters more than a single number.

Records	Naive queue (ms)	Circular queue (ms)	Naive dispatcher (ms)	Heap dispatcher (ms)	End-to-end (ms)
1,000	1.580	0.021	5.926	0.166	0.530
10,000	30.480	0.460	154.630	2.129	7.504
100,000	3,731.709	2.932	15,742.819	14.686	47.052

At 100,000 records	Measured comparison
Queue	$3,731.709 / 2.932 = \text{about } 1,273\text{x}$ faster for the circular queue
Dispatcher	$15,742.819 / 14.686 = \text{about } 1,072\text{x}$ faster for the heap
Designed pipeline	100,000 records completed in 47.052 ms without losing a record

Does the growth match the theory?

Mostly yes. The shifting queue grows sharply because draining n items performs roughly n squared over two moves. The circular queue only changes head and tail indexes, so it stays close to linear for a complete enqueue-and-drain run. Small timings are noisy because the work takes less than one millisecond, but the separation becomes clear at 100,000.

The sorted-array dispatcher also becomes expensive as each insert may shift many records. The heap grows much more slowly because its height is logarithmic and the burst begins with a linear build-heap. The complete designed pipeline rises from 0.530 ms to 47.052 ms when the input rises by 100 times, which is consistent with near-linear $n \log n$ work rather than quadratic work.

Other checks

- WP1 forced a collision, updated one record from version 1 to 3, returned null for an unknown ID, and completed 100,000 lookups.
- WP4 restored v3 to v2 and then v1, and an empty pop threw the required exception.
- WP5 triggered LL, RR, LR, and RL rotations. Its range query returned results in ascending time order.
- WP6 treated exactly 23:59:00.000 as on time and found the first strictly late row by binary search.

Conclusion

The naive versions were useful because they gave me a working reference and made the cost visible. The designed versions keep the same behavior but scale much better. For deadline night, I would keep separate chaining, reject-new circular intake, the complete heap priority key, AVL timestamps, and merge sort. These choices are simple enough for me to explain and their measured results agree with the expected Big-O costs.